

ReadMe Project -03

GROUP MEMBERS:(SAME AS LAST TIME)

- (1.) Jaya Sankar Sailesh Gupta Thondepu - U24924472
- (2.) Sai Keerthana Todupunoori - U40159095
- (3.) Aditya Madabhushi - U16072229

SYSTEM ENVIRONMENT:

Operating System: Ubuntu 22.04.3 LTS
Processor: Intel Core i5
Software used: xv6

Part 1 : Understanding sbrk():

To delete the page allocation we removed the growproc(n) call and added a line myproc() - > sz += n which increments the process logical space but since we removed the physical memory allocation no physical memory is allocated to the increased logical space.

1) Working of page allocation in xv6:

In xv6 page allocation is done immediately when the process requests more memory through sbrk(int n) call. sbrk(int n) function call increases the logical space of the system then we use growproc(n) to allocate physical memory pages to the new expanded logical space. In simple terms growproc(n) extends the heap by n bytes and ensures that these extended bits are mapped to physical memory so that when we access these extended memory pages there is no page fault.

2) Reason for the system break:

So the updated sbrk() system call does not have any memory allocation so when a process tries to access newly allocated memory the kernel throws a page fault. Which is of the form : "pid 3 sh: trap 14 err 6 on cpu 0 eip 0x1240 addr 0x4004--kill proc" because there is no physical page that backs the extended logical space.

3) Significance of the sbrk():

The sbrk() syscall holds a pivotal role within xv6 for a multitude of reasons, all of which contribute significantly to the operating system's approach to handling memory:

- **Dynamic Memory Control:** This syscall acts as a fundamental tool for processes to dynamically adjust their heap size, which is essential for operations like dynamic memory allocation and deallocation (notably seen in the use of malloc() and free() in the user-space programming).
- **Ensuring Process Separation:** By centralizing memory allocation within a controlled system call layer, xv6 effectively maintains a barrier between processes. This safeguard prevents any given process from encroaching on the memory space allocated to another, reinforcing the security and integrity of process isolation.

PART 2:

Lazy Allocator Implementation in xv6

Lazy allocation delays the allocation of physical memory until it is actually accessed, which is a critical feature for efficient memory management in operating systems. This approach minimizes the use of physical memory by allocating pages only upon actual access attempts that result in page faults.

Code Explanation

- **Page Fault Location:** Upon encountering a page fault, the handler first prints the faulting address, retrieved via `rcr2()`, which contains the linear address that triggered the page fault.
- **Address Alignment:** The faulting address is then aligned down to the nearest page boundary using `PGROUNDDOWN(rcr2())`. This ensures that any memory allocation or mapping aligns with the start of a page, adhering to the paging mechanism's requirements.
- **Memory Allocation:** `kalloc()` is called to allocate a new page of physical memory. If `kalloc()` returns 0, it indicates that the system has run out of free physical memory, prompting an "out of memory" message and the termination of the current process to prevent further illegal memory access.
- **Memory Initialization:** The newly allocated page is then cleared (filled with zeroes) using `memset()`, preparing it for safe use by the process. This step is crucial to eliminate residual data that could lead to security vulnerabilities or unpredictable behavior.

PART 3:

Locality-Aware Allocator Implementation in xv6

Unlike traditional lazy allocation, which only allocates memory upon access, the locality-aware approach preemptively allocates adjacent pages, anticipating near-future memory accesses. This strategy aims to optimize memory access patterns that exhibit spatial locality, reducing the overhead associated with handling page faults.

Code Breakdown

- **Initial Setup:** Upon detecting a page fault, the handler logs the event and determines the faulting address using `rcr2()`. It then aligns this address to a page boundary to ensure page-aligned operations.
- **Preemptive Allocation Loop:** The core of this strategy is a loop that preemptively allocates not just the faulting page but two additional pages. This is based on the assumption that subsequent accesses are likely to be close to the initial faulting address, a common pattern in applications.

- **Page Allocation and Initialization:** For each page, `kalloc()` is invoked to allocate physical memory. If allocation fails, an error is logged, and the process is terminated to prevent illegal memory access. Each newly allocated page is zero-initialized to ensure a clean state before usage.
- **Address Mapping:** The allocated pages are mapped into the process's virtual address space at calculated addresses. This step is crucial for making the newly allocated memory accessible at the expected virtual addresses.
- **Error Handling:** The implementation includes robust error handling for both allocation and mapping failures, ensuring system stability by terminating processes that encounter unrecoverable errors.

PART 4:

B: Implementation of Lazy and Locality aware allocators:

Context and Purpose

In xv6, a page fault occurs when a process tries to access a portion of its virtual address that don't have a corresponding physical page allocated. The operating system's response to this event can significantly impact performance, especially for applications with specific memory access patterns.

Modifications to the Page Fault Handler:

So what we did was we updated the default page fault handler in `trap.c`. we have used `ALLOCATOR` flag to change between `LAZY ALLOCATOR` and `LOCALITY AWARE ALLOCATOR`

Lazy Allocation Strategy

- **Trigger:** Activated when `ALLOCATOR_LOCALITY` is not defined.
- **Operation:** On a page fault, the handler allocates exactly one page of physical memory.
 - It calls `kalloc()` to obtain a new page.
 - Using `PGROUNDDOWN(rcr2())`, it calculates the start of the page that caused the fault, ensuring that the new page is aligned to a 4KB boundary.
 - It then maps this new page to the faulting process's address space at the calculated address, employing `mappages()`.
- **Rationale:** This strategy embodies the principle of allocating memory only as needed, minimizing the upfront allocation of physical memory and thereby potentially conserving resources. However, it may result in a higher number of page faults for workloads that access many contiguous pages of memory.

Locality-Aware Allocation Strategy

- Trigger: Activated when ALLOCATOR_LOCALITY is defined at compile time.
- Operation: Upon encountering a page fault, the handler allocates and maps three consecutive pages starting at the fault address.
 - Similar to the lazy strategy, it begins by calculating the faulting page's start address.
 - It then enters a loop where, for three iterations, it allocates a new page with `kalloc()` and maps each page into the process's virtual address space. The mappings cover the faulting page and the next two pages sequentially.
- Rationale: This preemptive strategy is designed to take advantage of spatial locality, where a program is likely to access memory locations near those it has already accessed. By allocating and mapping additional pages in advance, it aims to reduce the total number of page faults experienced by a process, potentially improving performance for certain workloads.

Debugging and Verification

To facilitate debugging and performance evaluation, print statements are strategically placed to indicate which allocation strategy is being used and the memory address at which a page fault occurred. These outputs serve as immediate feedback on the behavior of the modified page fault handler under different workloads.

- For the lazy allocator: A message is printed each time a single page is allocated in response to a fault.
- For the locality-aware allocator: A message is printed indicating that multiple pages are being allocated preemptively in response to a single fault.

Considerations

- Memory Usage: While the locality-aware strategy can improve performance by reducing page faults, it may also increase memory usage by allocating pages that a process may not end up accessing.

C. WORKING OF ALLOCATORS:

We see the same page fault locations for various commands, even with the locality-aware allocator enabled, suggesting that the actions triggered by these commands involve accessing the same virtual memory addresses that are not currently mapped to physical pages. We were not able to exactly differentiate between the functionality of lazy and locality aware using these simple commands so we wrote a user program which differentiates them clearly.

User program (test_pages):

This code is a user-space program designed for xv6 to specifically test how the operating system's page fault handler responds to memory access patterns, particularly in the context of comparing lazy allocation with locality-aware allocation.

Code Breakdown

Memory Allocation: The program begins by dynamically increasing the process's data segment to accommodate a large array. This is achieved through the `sbrk(ARRAY_SIZE)` system call, which adjusts the program's break point by `ARRAY_SIZE` bytes. `ARRAY_SIZE` is defined as the product of the number of pages (`NUM_PAGES`) and the size of each page (`PAGE_SIZE`, typically 4KB in xv6), thus ensuring enough space for the array.

Initial Memory Access (Lazy Trigger): The first loop iterates over the newly allocated space, accessing the first byte of each page (`array[i] = 'a';`). Since this memory has been allocated but not yet accessed, these accesses are likely to trigger page faults. In the context of lazy allocation, each fault results in the allocation and mapping of a single new page at the accessed address.

Subsequent Memory Access (Locality Test): The second loop is designed to access memory within each page more densely (`step = PAGE_SIZE / 4`). By writing to the array at these smaller intervals, the program tests whether accessing nearby memory locations triggers additional page faults. This is particularly relevant for the locality-aware allocation strategy, which pre-allocates adjacent pages in anticipation of such accesses.

Output:

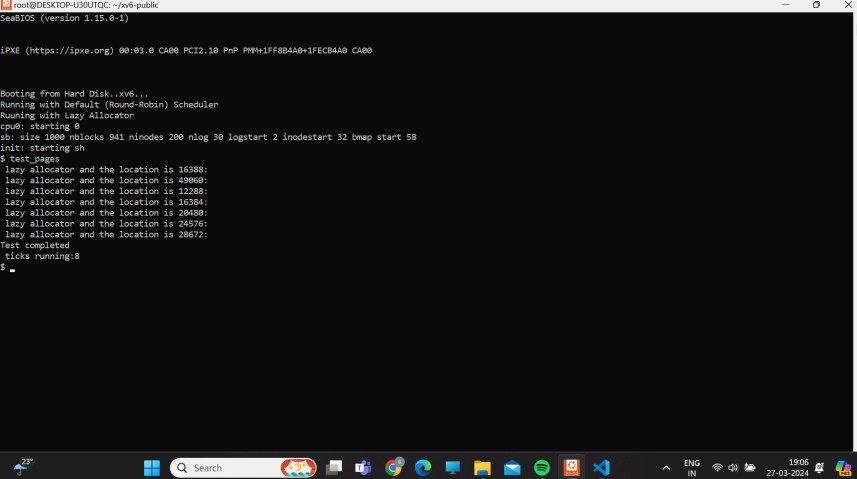
- **Number of Page Faults:** The locality-aware allocation strategy results in fewer page faults being logged for the same set of memory accesses, as adjacent pages are pre-allocated. This directly contrasts with the lazy allocation strategy, where each new page accessed triggers a separate fault.
- **Performance:** we have used ticks running (from a previous project) to calculate the amount of time the process is running. As expected, Locality-aware allocator ran for less time when compared to Lazy.

Explanation of the outcome:

- **Lazy Allocation:** With lazy allocation, each access that triggers a page fault results in the allocation of exactly one page. The initial loop should trigger a page fault for each page accessed, with the OS allocating precisely one page for each fault. The subsequent accesses within each page should not cause additional faults, assuming no other factors prompt them.
- **Locality-Aware Allocation:** This strategy anticipates that subsequent accesses will occur near the initially accessed address and allocates additional pages preemptively. The initial accesses in the first loop should trigger fewer page faults overall, as multiple pages will be allocated at once. The second loop's accesses are less likely to cause new

page faults, provided they fall within the range of pages pre-allocated during the initial faults.

```
$ test_pages
locality allocator and the location is:16388
locality allocator and the location is:49860
locality allocator and the location is:12288
locality allocator and the location is:24576
Test completed
ticks running:4
$
```



```
root@DESKTOP-JUNUTOC: ~/xv6-public
SeaBIOS (version 1.15.0-1)

IPXE (https://ipxe.org) 00:03:0 CA00 PCI2.10 PNP PMM+1F8B4A0+1FECB4A0 CA00

Booting from Hard Disk..xv6...
Running with Default (Round-Robin) Scheduler
Running with lazy Allocator
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
unit: starting 25
$ test_pages
lazy allocator and the location is 16388:
lazy allocator and the location is 49860:
lazy allocator and the location is 12288:
lazy allocator and the location is 16384:
lazy allocator and the location is 28488:
lazy allocator and the location is 24576:
lazy allocator and the location is 28672:
Test completed
ticks running:8
$ _
```